DressMe agents: study guide

BO1 Personalised recommendations · BO2 Time optimisation · Racem Njeh · ESPRIT 2026

Everything about the agents, explained from zero: what an agent is, how the agents call each other, when the LLM is used and when it is skipped, the memory, the tools, the techniques, and every agent of every idea with its workflow and an example. Everything described here is in the code (`backend/shared/idea_agent.py`, `backend/bo/*/agents.py`) and was run.

Part A · The big picture

1. What an "agent" is in DressMe
2. Three levels: BO agents, idea agents, tools
3. The messages agents send each other
4. How agents find and call each other (agent bus)

Part B · The techniques

5. The LLM and its 4 jobs
6. When the LLM runs and when it is skipped
7. Prompt engineering (the real prompts)
8. Guardrails (keyword rules)
9. RAG and grounding
10. Memory (5 kinds)
11. Pretrained models and optimisation
12. The tools (module agents)

Part C · Every agent, one by one

13. BO1 idea agents (9)
14. BO2 idea agents (7)
15. The two BO agents (orchestrators)

Part D · The BOs as a whole

16. BO1 as a system · 17. BO2 as a system
18. Three full walkthroughs
19. Errors, questions, limits · 20. Glossary

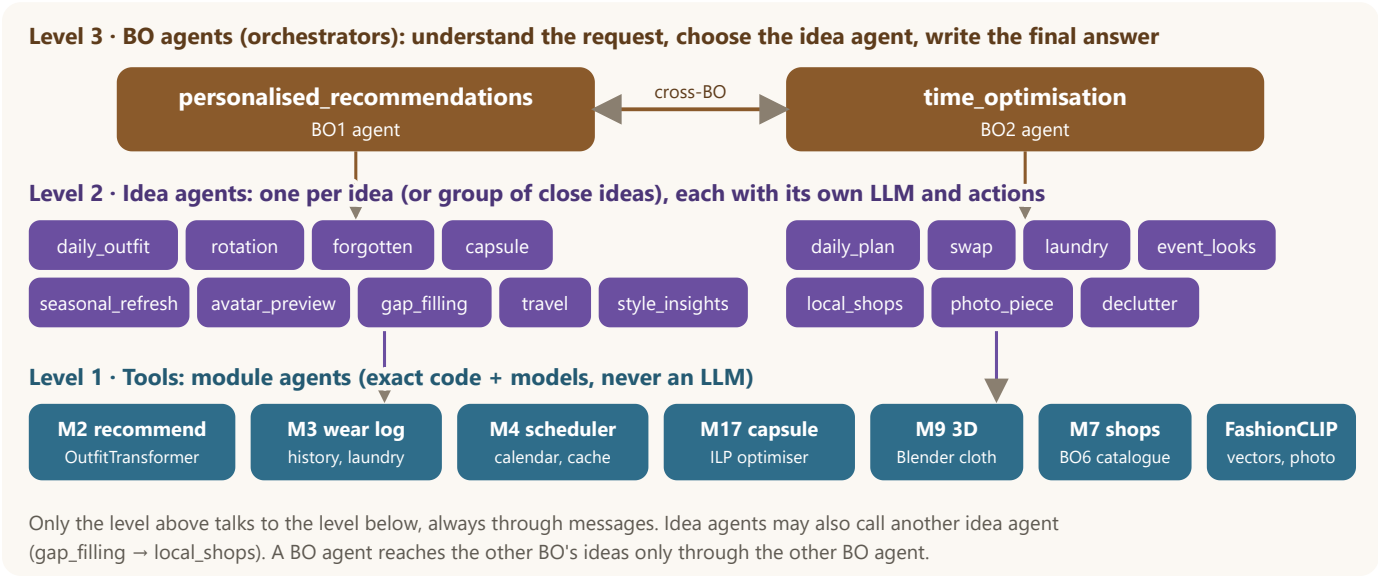
1. What an "agent" is in DressMe

A normal function does one fixed thing. An **agent** receives a request, **decides** what to do, **uses tools** to do it, **remembers** things, and **answers**. In DressMe every agent has the same four parts:

Part	What it is in DressMe	Analogy: a personal stylist
Brain (LLM)	Qwen2.5 3B, a small language model running on the laptop. It reads a sentence, decides which action to take and with which arguments, and writes the answer in English / French.	The stylist understands what you say and talks back.
Actions (workflows)	A few fixed procedures written in Python, e.g. `recommend`, `swap`, `put` in the laundry. Each action is a step-by-step workflow.	The things the stylist knows how to do.
Tools	The team's modules (M2 recommendation, M3 wear log, M4 scheduler, M7 shops, M9 3D, M17 capsule) and models (FashionCLIP). They do the exact work.	The stylist's catalogue, calendar, notebook, and the shops they know.
Memory	The conversation, a shared "blackboard" of the session (last outfit…), the user's taste profile, the wear history (M3) and the wardrobe vectors.	The stylist remembers what you wore, what you hate, what they proposed 2 minutes ago.

> **The most important idea of the whole design:** the LLM **understands and explains**; the tools **decide the clothes**. The LLM never invents an outfit: outfits come from M2 (OutfitTransformer), capsules from M17 (optimisation), history from M3. That is why a request that is already precise (a button) can skip the LLM completely and still get the same result.

2. Three levels: BO agents, idea agents, tools



Only the level above talks to the level below, always through messages. Idea agents may also call another idea agent (gap_filling → local_shops). A BO agent reaches the other BO's ideas only through the other BO agent.

- **Why three levels?** A small LLM (3B) is bad at choosing among ~40 actions at once (we measured 4/7 correct). Splitting the choice in two (first the idea agent, then the action inside it) gives each LLM a short list: 9/9 correct on the test conversation.
- **Why are the tools not LLM agents?** They must be exact and fast (a database, an optimiser, a neural recommender). They are "agents" only in the sense that they accept messages (`{action, payload}`) and announce what they can do.
- **16 idea agents** cover the 30+ BO ideas: close ideas share one agent (e.g. `daily_plan` = outfit of the day + tomorrow's outfit + notification + widget).

3. The messages agents send each other

All agents speak the protocol of BO8 (the team's orchestrator), file `shared/agent_card.py`, copied unchanged.

`AgentCall` (the request)

```
{
 "user_id": "demo_f",
 "session_id": "s1",        ← groups the turns
 "language": "en",
 "message": "change the shoes",   ← a sentence, OR
 "payload": {               ← a precise order
   "action": "swap",
   "args": {"slot": "shoes"},
   "reply": false           ← no sentence needed
 }
}
```

`AgentResult` (the answer)

```
{
 "agent": "swap",
 "question": null,   ← set if info is missing
 "next_actions": [], ← buttons to offer next
 "result": {
   "action": "swap",
   "reply": "...",   ← LLM sentence (if asked)
   "data": {"before": [...], "after": [...]},
   "steps": [...],   ← the full trace
   "model": "qwen2.5:3b"
 }
}
```

Every agent also has a **card** (its ID card): name, owner BO, role, ideas covered, *when to use / when not to use*, skills, actions with their parameters, and *handoff* buttons. The BO agents' LLMs choose an idea agent by **reading these cards**. BO8 can read them too (`GET /agents`).

A **tool** (module agent) uses a simpler message: `{"action": "swap", "user_id": "...", "payload": {...}, "sender": "swap"}` → `{"agent", "action", "ok": true, "result": {...}}`. The payload is validated with the module's own request model (Pydantic), so a wrong argument is refused cleanly.

4. How agents find and call each other: the agent bus

- `shared/agent_bus.py` discovers every agent at start-up: each `modules/*/agent.py` (the tools) and each `bo/*/agents.py` (the idea agents and BO agents). It is a directory: *name* → *agent*.

- Inside a workflow an agent calls another one with one line: `await ctx.call("m02_recommendation", "swap", {...})` or `await ctx.call("local_shops", "find", {...})`.
- `ctx.call` looks at who is called: an idea agent receives an `AgentCall` (with `reply: false`, so it does not write a sentence); a tool receives `{action, payload}`.
- Each call is recorded as a **step** (`agent` or `tool`, with its time and result); the called agent's own steps are nested inside (`sub_steps`). That is how the demo pages draw the "who called whom" tree.
- From outside (the app, BO8, n8n): `POST /bo/chat` or `POST /agents/{name}` with the same AgentCall.

5. The LLM and its 4 jobs

Model	**Qwen2.5 3B Instruct** (Alibaba, open weights), served by **Ollama** on the laptop (`qwen2.5:3b`, ~1.9 GB, RTX 2050 4 GB). No internet, no API key, no user data leaves the machine. 3–9 s per call. Code: `shared/local_llm.py`. Settings: temperature 0.2 (stable answers), context 8192 tokens.
JSON mode	For decisions the LLM is forced to answer in JSON (*constrained decoding*: Ollama only lets it produce valid JSON). So the code can read `{"action": "swap", "args": {"slot": "shoes"}}` reliably instead of parsing free text.

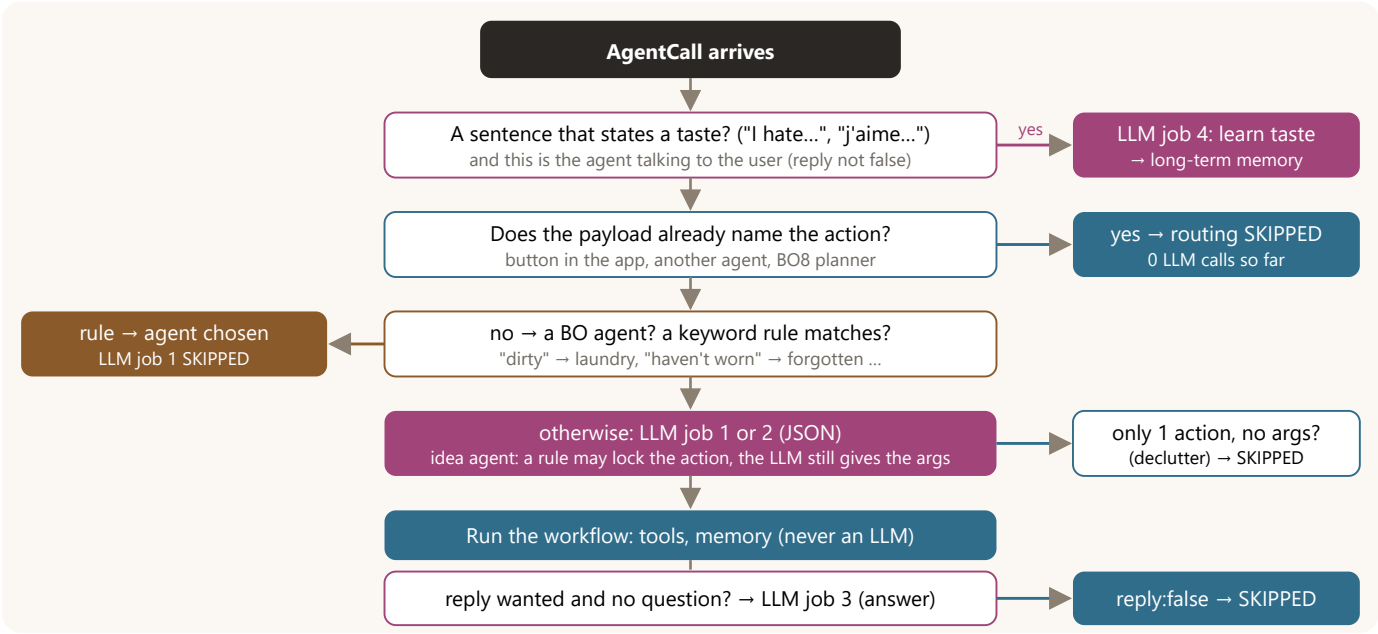
The LLM has exactly **four jobs**. Every LLM call appears in `result.steps` with its name and time.

#	Job	Who	Input → output	Step name
1	**Choose the idea agent**	BO agent	sentence + the idea agents' cards → `{"agent": "daily_outfit"}`	`llm_route`
2	**Choose the action + extract the arguments**	idea agent	sentence + its actions + taste profile + session state → `{"action": "recommend", "args": {"day": "tomorrow", "occasion": "work"}}`	`llm_route`
3	**Write the answer**	the agent that talks to the user	FACTS (tool results) + conversation → 2–4 sentences in the user's language	`llm_answer`
4	**Learn a taste**	the agent that talks to the user	"I hate yellow" → `{"kind": "dislikes", "value": "yellow"}` saved in the profile	`memory_write`

> **What the LLM never does:** choose the clothes, compute a score, a price, a date difference, a cost per wear, or read the database. All of that is code and tools. The LLM only *routes* (jobs 1, 2), *explains* (3) and *listens* (4).

6. When the LLM runs, and when it is skipped

This is decided in `IdeaAgent.run` (one place for all agents). Follow the request from top to bottom:



Situation	Example	LLM calls
Sentence to a BO agent, no rule	"What should I wear tomorrow for work?" → BO1	**3** (choose agent · choose action+args · answer) ≈ 14 s
Sentence to a BO agent, rule matches	"my black t-shirt is dirty" → BO2	**2** (rule picks laundry; LLM extracts "black t-shirt"; answer)
Sentence stating a taste	"I hate yellow, what do I wear?"	**+1** (learn taste), saved for every future request
Sentence to an idea agent	"change the shoes" → swap	**2** (action+args · answer)
Sentence to declutter	"what can I get rid of?"	**1** (answer only: one action without arguments, nothing to choose)
Button in the app	"Change shoes" → `{"action":"swap","args":{"slot":"shoes"},"reply":false}`	**0**, < 1 s
Agent → agent inside a workflow	BO2 agent → BO1 agent → forgotten	**0** in the called agents (the caller gives the action; only the top agent writes)
Agent → tool	swap → M2 swap	**0**, never
Missing information	"plan my trip" (no city)	action chosen by LLM, then the workflow returns a **question** ("Where are you going?"); no answer LLM needed

Why skipping is correct, not a shortcut. Buttons are how the real app works (the screen knows what the user tapped), so asking an LLM to "understand" a tap would waste 6–20 s and add a chance of error. The chat is for free text: "Quick natural-language request" is a BO2 idea, and it is exactly the BO2 agent with its LLM. Same agents, same tools, same memory, two entrances.

7. Prompt engineering: the real prompts

Router prompt (LLM job 2, every idea agent)

```
You are the {name} agent of DressMe ({role}).                    ← role prompt, built from the agent's card
Choose the ONE action that answers the user's message and fill its arguments.
Actions:
- recommend: outfits for a day, occasion and mood args: {"day": "today | tomorrow | YYYY-MM-DD", ...}
- remix: a different outfit from the last one ...                ← the agent's own short list (2–8 actions)
User taste (long-term memory): {"likes": [...], "dislikes": ["yellow"]}       ← memory injected
Session state: {"last_outfit": [...], "last_day": "2026-10-23", ...}           ← blackboard injected
Reply with JSON only: {"action": "<one action name>", "args": {...}}.        ← output format + JSON mode
If the message is unclear, choose the most useful action with default arguments. Never invent a piece name.
```

The last 4 turns of the conversation are added before the user's message, so follow-ups ("and for Saturday?") are understood.

BO agent prompt (LLM job 1)

```
You are the personalised_recommendations agent of DressMe (...role...)
Choose the ONE agent that should handle the user's message.
Agents:
- daily_outfit: Proposes the outfit of a day ... Examples: What do I wear today / tomorrow?; Give me another one; ...
- time_optimisation:laundry: Keeps track of what is in the laundry ... Examples: My jeans are dirty; ...   ← partner
BO's ideas
Reply with JSON only: {"agent": "<agent name exactly as listed>"}
```

Writer prompt (LLM job 3)

```
You are the {name} agent of DressMe ({role}), talking to the user.
Answer their last message using ONLY the FACTS (results of your tools). Name pieces exactly as in FACTS.
If FACTS has recommended_outfit, present exactly those pieces together as one outfit (alternative_outfit is a backup).
Never invent a piece, a price, a date or a score: copy every number exactly as written in FACTS and never
compute a new one. 2 to 4 short sentences. Write in {language}.
FACTS: {"action": "recommend", "recommended_outfit": ["black t-shirt", "blue skirt", "black shoes"], ...}
...conversation...
user: What should I wear tomorrow for work?  (Answer in English.)      ← language repeated at the very end
```

Technique	Why it is there (each one fixed a real problem seen in testing)
Role prompt from the card	The same base class serves 18 agents; each gets its own identity and action list automatically.
JSON mode	Free-text decisions were unreadable ("I think you should use swap..."). JSON is always parseable.
Short action list per agent	~40 actions at once: 4/7 correct; 2–8 actions per agent: 9/9.
FACTS only, recommended first, one alternative, no ids	The 3B model mixed pieces of different outfits when it saw all of them, and copied ids into the answer.
"Copy every number, never compute"	The model invented a cost per wear once. Numbers are now computed by code, copied by the LLM.
Language in the last line	The model answered French questions in English; an instruction placed last is followed best.
Examples without real piece names	With "red dress" in an example, the model proposed the red dress even when it was in the laundry.
Temperature 0.2	Same request → same decision (reproducible demo).

8. Guardrails: keyword rules

A 3B model sometimes picks the wrong agent. When the wording is unambiguous, a **keyword rule** decides instead. Two kinds:

- **BO agent rules: replace the LLM.** Example BO2: `"dirty", "laundry", "lessive"` → laundry; `"haven't worn", "forgot", "oubli"` → BO1's forgotten; `"wedding", "mariage", "interview"` → event_looks; `"photo"` → photo_piece. The step shows `rule_route` and LLM job 1 is skipped.
- **Idea agent rules: lock the action, keep the LLM.** Example laundry: `"done", "clean", "propre"` → back; `"dirty", "wash", "sale"` → put. The LLM still runs to extract the arguments ("which pieces?"); the step shows `how: "rule"` or `"llm+rule"` (both agreed).
- **Argument guard:** the router keeps only the arguments that belong to the chosen action (the 3B model once filled `pieces`, an argument of *build*, while choosing *recommend*), and key workflows read the sentence's words as a backup.
- **Fallback guard:** if the LLM answers an action that does not exist, the agent uses its keyword fallback or its default action (`how: "guard"`), never crashes.

Result on the 9-message test conversation (English + French, cross-BO): 4/7 correct before guardrails and two-stage routing, 9/9 after.

9. RAG and grounding

RAG (retrieval-augmented generation) = before acting or answering, the agent **retrieves** the facts it needs, and the LLM only uses those facts. In DressMe:

What is retrieved	From	Used for
The user's wardrobe (each piece + its FashionCLIP vector)	M9 3D wardrobe (the user's photos) if ≥ 2 pieces, else an M2 demo wardrobe (`shared/wardrobe.py`, step `wardrobe`)	every outfit tool; names of pieces in answers
Pieces closest to the request	FashionCLIP text space (cosine similarity) + pieces named in the sentence (`retrieve()`)	"build an outfit with my *jeans*" → the right piece id
Wear history, laundry, lent	M3	rotation, forgotten, laundry, declutter, palette, drift, lookbook
Calendar, events, cached outfit	M4	occasion of the day, events, widget
Weather	Open-Meteo (through M2)	travel, seasonal refresh, daily outfit
Taste profile, session state	agent memory	router prompt, dislikes excluded

Grounding = the writer LLM sees *only* the FACTS dictionary built by `writer_facts()` (recommended outfit, why, one alternative, numbers), never ids or score internals. So it cannot talk about a piece the user does not own.

10. Memory: 5 kinds

Kind	What is stored	Who writes	Who reads	Where
Short-term · conversation	The chat turns of a session (user + assistant)	the top agent, after each sentence turn	every LLM call (last 4 turns replayed)	`.agent_memory/<user>/chat_<session>.json`
Short-term · session blackboard	`last_outfit`, `last_outfits`, `last_day`, `last_occasion`, `capsule`, `last_photo`	workflows (`ctx.remember(...)`)	workflows (`ctx.state`) + router prompt	`state_<session>.json`
Long-term · taste profile	likes / dislikes / notes	LLM job 4 ("I hate yellow")	router prompt; dislikes become hard filters (`exclude_item_ids`)	`profile.json`
Episodic	what was worn when, laundry / lent status, posted looks	M3 (through laundry, daily_plan.wear)	rotation, forgotten, palette, drift, lookbook, declutter, and M2 (laundry excluded)	M3 database (SQLite / Postgres)
Semantic	every piece as a FashionCLIP vector (image + text)	M9 / M1 when a piece is added	every outfit tool, retrieval, duplicates	M9 wardrobe / M2 demo catalogue

The blackboard is how agents cooperate without talking directly. Example in one session:

1. **daily_plan.today** (BO2) writes `last_outfit = [red dress, burgundy shoes]`.
2. **swap** (BO2) reads it, asks M2 for new shoes, writes `last_outfit = [red dress, olive shoes]`.
3. **daily_outfit.why** (BO1) reads `last_outfits` and asks M2 to explain that exact outfit.
4. **avatar_preview** (BO1) reads `last_outfit` and dresses the 3D avatar with it.
5. **laundry.put** (BO2) → M3: the dress is now "laundry", so M2 will not propose it tomorrow (episodic memory).

Four different agents from two BOs, one shared memory of "the outfit we are talking about".

Long-term example: "I hate yellow" → profile `dislikes: ["yellow"]` → the next day, daily_outfit sends M2 `exclude_item_ids = [yellow pants]`: no yellow piece can be proposed, by any agent that uses `_recommend` (daily_outfit, rotation, seasonal_refresh, travel).

11. Pretrained models and optimisation (nothing trained)

Model / method	What it does	Used by
OutfitTransformer (pretrained)	Reads the pieces of an outfit (their vectors) and outputs a compatibility score (do they go together?). AUC 0.95 on the benchmark. M2 generates candidate outfits and ranks them with it, plus context (occasion, weather, rotation).	M2 → daily_outfit, swap, forgotten, travel, event_looks, photo_piece…
FashionCLIP (pretrained)	Turns an image or a text into a 512-number vector; similar things get close vectors. **Zero-shot**: to know the type of a photo, compare its vector to the vectors of "a photo of a t-shirt", "…of jeans"… and take the closest: no training needed.	retrieval (RAG), photo_piece (type, colour, duplicates), declutter (near-duplicates), M7 search
BiRefNet (pretrained)	Removes the background of clothing photos.	M9 3D pieces
Integer linear programming (SciPy milp / HiGHS)	Chooses the N pieces that make the most good outfits, exactly (optimal). Greedy comparison: 89 outfits (ILP) vs 47 (greedy) in the demo.	M17 → capsule
Greedy reuse	For each trip day, among 6 good outfits, take the one reusing the most already-packed pieces → fewest pieces in the suitcase.	travel
Blender cloth physics	Drapes the 3D garments on the avatar.	M9 → avatar_preview
Qwen2.5 3B (pretrained LLM)	Routing, answers, taste extraction. No fine-tuning: prompt engineering + JSON mode + guardrails were enough.	all agents

12. The tools (module agents)

Tool	Owner	Actions the idea agents use
m02_recommendation	Racem (M2)	`recommend` (outfits for a context) · `remix` (different from a base outfit) · `complete` (around pinned pieces) · `swap` (replace one slot) · `score` (score + breakdown of a given outfit)
m03_wear_log	Racem (M3)	`stats` · `forgotten` · `history` · `states` · `status` (laundry / lent / available) · `laundry_done` · `worn` · `recap`
m04_scheduler	Racem (M4)	`today` · `tomorrow` (cached, computed at 05:00 / 20:00) · `approve` · `change` · `wear` · `week` · `calendar` · `event_add` · `widget` · `notifications` · `run`
m17_capsule	Racem (M17)	`capsule` (ILP) · `plan`
m09_avatar_3d	Racem (M9)	`items` (the user's pieces + vectors) · `pieces` · `outfit3d` · `render` · `set_body`
m07_local_catalogue	BO6 (copied unchanged)	`search` : Tunisian shops, FashionCLIP + BM25 + attributes, budget TND, city
FashionCLIP / Open-Meteo	pretrained / public API	photo encoding, zero-shot · weather forecast

13. BO1 idea agents (Personalised recommendations)

How to read each card: **Actions** = what it can do (arguments in brackets) · **Workflow** = the exact steps of the main action · **Memory** = what it reads (R) and writes (W) · **LLM** = when its LLM runs · **Example** = a real run.

daily_outfit

Daily outfit + Remix · Outfit builder · "Why this outfit"

Actions	`recommend(day, occasion, mood)` · `remix()` · `build(pieces, occasion)` · `why(n)`
Workflow recommend	1. day and occasion from the LLM's arguments, backed up by the sentence's own words ("tomorrow", "demain", "work"), because a 3B model sometimes fills the wrong arguments · 2. **M4 calendar** → occasion of that day (work / casual / event, from the user's events) · 3. load wardrobe (RAG) · 4. remove disliked pieces (long-term memory) · 5. **M2 recommend** → top 3 outfits with weather · 6. save them on the blackboard · 7. offer the button "Show it on my avatar".
Other actions	**remix**: M2 remix of `last_outfit` (shares at most one piece) · **build**: names → piece ids (resolve), M2 complete around them; asks "Which of your pieces?" if none found · **why**: M2 score of the last outfit → score breakdown (colour, style, weather, rotation…) → top 3 reasons.
Tools	M4 calendar · M2 recommend / remix / complete / score
Memory	R: dislikes, last_outfits, last_occasion · W: last_outfit(s), last_day, last_occasion
Rules	"why / pourquoi" → why · "another / remix / autre" → remix · "with my / avec mon" → build · "wear / tenue" → recommend
Example	"What should I wear tomorrow for work?" → work day, 22.7 °C → *red dress* + *burgundy shoes* (0.83), alternative black t-shirt + blue skirt + black shoes.

LLM: sentence → picks the action + day/occasion/mood, then writes the answer (for *why* it turns the breakdown into a sentence). Button → 0 LLM calls.

rotation

Rotation-aware penalty

Actions	`report()` · `fresh_outfits(days, occasion)`
Workflow fresh	1. wardrobe · 2. **M3 stats** → days since each piece was worn · 3. list pieces worn in the last N days (default 7) · 4. **M2 recommend** excluding them + rotation penalty window · 5. blackboard.
Workflow report	M3 stats → the 5 most worn pieces in 30 days, how many pieces were used, rotation rate = used / total.
Memory	R: M3 history (episodic), dislikes · W: last_outfit(s)
Example	report → white shoes ×10, black shorts ×6 in 30 days; fresh outfits leave out 7 recently worn pieces.

LLM: sentence → report or fresh_outfits + days, then the answer. Button → 0.

forgotten

Rediscover forgotten pieces

Actions	`rediscover(days)` (default 45)
Workflow	1. wardrobe · 2. **M3 forgotten** → pieces not worn for ≥ N days · 3. keep available ones (not in laundry) that can start an outfit (top / bottom / dress / outerwear) · 4. the most forgotten = "bring back" · 5. **M2 complete** around it → 2 outfits · 6. blackboard.
Memory	R: M3 · W: last_outfit(s)
Example	light blue polo shirt (never worn) + navy pants + white shoes (0.74).

LLM: sentence → extracts "days" (e.g. "not worn for 2 months" → 60), then the answer. Often reached from BO2 by the rule "haven't worn". Button → 0.

capsule

Capsule wardrobe mode

Actions	`build(size, season)` · `plan(days)`
Workflow build	1. wardrobe with vectors · 2. **M17 capsule** (ILP over M2 compatibility scores, method "ilp") → the N pieces making the most outfits · 3. save `capsule` on the blackboard.
Workflow plan	reads `capsule` (asks "Build a capsule first" if none) → **M17 plan** → one outfit per day from the capsule.
Example	8 pieces (autumn): grey long-sleeve top, grey t-shirt, white shirt, blue pants, yellow pants, grey shoes, white shoes, white jacket → 24 outfits (16 casual, 8 work). ~16 s (optimiser).

LLM: sentence → size/season or days, then the answer. The capsule itself is chosen by the optimiser, never by the LLM. Button → 0.

seasonal_refresh

Seasonal wardrobe refresh

Actions	`refresh(city)` (default Tunis)
Workflow	1. Open-Meteo forecast for days 0, 2, 4, 6 → mean temperature · 2. season: ≥ 26 °C summer, ≥ 16 °C spring/autumn, else winter · 3. **M3 forgotten** (60 days) → pieces to take out · 4. **M2 recommend** with a bonus for forgotten pieces → looks.
Example	mean 21.2 °C → spring/autumn; take out light blue polo shirt, grey long-sleeve top…

LLM: sentence → city, then the answer. Button → 0.

avatar_preview

3D avatar preview of the daily outfit

Actions	`preview(animate)`
Workflow	1. wardrobe: must be the user's own 3D wardrobe (else "add photos first") · 2. outfit = blackboard `last_outfit`, else **M4 today** · 3. keep only the user's 3D pieces · 4. **M9 outfit3d** (Blender cloth simulation, cached) · 5. link to the 3D viewer `/beta1/demo?wear=…`.
Memory	R: last_outfit (written by daily_outfit, swap, daily_plan…)

LLM: sentence → animate yes/no, then the answer. The 3D is Blender; the LLM only adds the sentence. Usually reached by the handoff button "Show it on my avatar" → 0.

gap_filling

Gap-filling suggestion

Actions	`suggest(budget_tnd, city)`
Workflow	1. count pieces per category vs an ideal mix (top 38 %, bottom 25 %, shoes 17 %, outerwear 10 %, dress 10 %) → biggest gap · 2. first missing neutral colour (black, white, navy, beige, grey) · 3. suggested piece, e.g. "black trousers" · 4. estimate of new outfits (e.g. new bottom × number of tops) · 5. **calls the local_shops agent** (agent → agent) with "black trousers Tunis 120 TND".
Example	missing slot bottom → black trousers → Moudda (Tunis) 69.9 TND.

LLM: sentence → budget/city, then the answer. local_shops is called with a structured call, so **its** LLM does not run. Button → 0.

travel

Travel outfit planner (BO1) + Packing outfits for a trip (BO2)

Actions	`plan(city, days, start, occasion)`
Workflow	1. no city → question "Where are you going, and for how many days?" · 2. for each day: Open-Meteo weather of the destination → **M2 recommend** 6 candidates with that weather · 3. **greedy**: pick the candidate reusing the most packed pieces (and not the same as yesterday) · 4. packing list = union of chosen pieces.
Example	"Je pars à Paris 3 jours" → 12 °C rain → 3 outfits, 6 pieces to pack, answer in French.

LLM: sentence → city, days, start; answer in the user's language. Button → 0.

style_insights

Personal colour palette · Style drift · Favourite looks / lookbook

Actions	`palette()` · `drift()` · `lookbook()`
Workflows	**palette**: main colour of each piece × (1 + times worn, from M3) → top 5 colours with share; complementary colour by rotating the hue 180° (colour harmony). **drift**: M3 history; colours worn in the last 30 days vs days 31–120 → rising / fading colours. **lookbook**: M3 history of a year → outfits worn most often, and posted ones.
Example	palette white 21 %, light blue 13 %, beige 12 %; drift: rising burgundy, blue.

LLM: sentence → which of the 3 (rules: "colour" → palette, "drift / chang" → drift, "favourite" → lookbook), then the answer. Button → 0.

14. BO2 idea agents (Time optimisation)

daily_plan

Outfit of the day (cached) · Tomorrow's outfit the night before · Notification · Widget

Actions	`today` · `tomorrow` · `approve(day)` · `change(day)` · `wear` · `week` · `notifications` · `widget`
Workflow today	1. **M4 today**: the outfit was **already computed at 05:00** (tomorrow's at 20:00) by the scheduler, so it is instant · 2. names of the pieces · 3. blackboard (last_outfit, last_day, last_occasion).
Others	**approve / change**: M4 lock or next option for the day (uses `last_day`) · **wear**: M4 → logged in M3 · **week**: 7 days, no piece repeated · **notifications**: M4 runs due jobs, returns the outbox · **widget**: compact data for the home screen.
Example	widget → {"summary": "red dress + burgundy shoes", "temp_c": 21.8, "deep_link": "dressme://today"} in 0.0 s.

> **LLM:** sentence → one of 8 actions (rules: "tomorrow / demain", "I take it", "another option", "I wore"...), then the answer. In the app these are buttons (Today, ✓ I take it, ↻ Another, Widget) → 0. The cached outfit itself never uses an LLM.

swap

One-tap swap of a single piece

Actions	`swap(slot)` : top | bottom | dress | outerwear | shoes
Workflow	1. current outfit = blackboard `last_outfit`, else M4 today · 2. slot from args or words ("jean" → bottom, "chaussure" → shoes) · 3. if the outfit has no such slot, take the first slot it has · 4. **M2 swap** → best 3 replacements keeping the rest · 5. blackboard updated.
Example	red dress + burgundy shoes → red dress + olive shoes (alternatives: black shoes, burgundy shoes).

> **LLM:** sentence ("change the shoes") → slot, then the answer. The "Change shoes" button → 0, 0.15 s.

laundry

Laundry / unavailable UI

Actions	`status` · `put(pieces)` · `back(pieces)` · `lend(pieces, to)`
Workflow put	1. piece names → ids (word matching on the wardrobe) · 2. none found → question "Which piece?" · 3. **M3 status** = laundry → from now on M2 never proposes them.
Others	**status**: M3 states → unavailable pieces + reminders · **back**: M3 laundry_done (all if no piece named) · **lend**: status lent + to whom.
Rules	"done / clean / propre" → back · "lent / prêté" → lend · "dirty / wash / sale" → put · "what's in" → status
Example	"my black t-shirt is dirty" → BO2 rule → laundry → rule locks put, LLM extracts "black t-shirt" → M3.

> **LLM:** extracts which pieces (the action is often locked by a rule), then the answer. Button "Outfit → laundry" sends the pieces itself → 0.

event_looks

Event countdown looks

Actions	`add(date, title, dress_code)` · `upcoming()`
Workflow add	1. date must be valid, else question "What is the event and on which date?" · 2. **M4 event_add** (with reminders) · 3. **M2 recommend** occasion event (classy if formal/festive) with that day's forecast · 4. days left = date − today (code, not LLM).
Workflow upcoming	M4 calendar 45 days → up to 3 events with countdown + 2 looks each.
Example	"I have a wedding on 2026-11-07" → event added, 16 days left, look blue dress + black shoes + navy jacket.

LLM: extracts date, title, dress code from the sentence, then the answer. Button "My events" → 0.

local_shops

Local shops shortlist for a missing piece

Actions	`find(query)`
Workflow	1. query (from args, or the sentence itself) · 2. **M7 search** (BO6's module, unchanged): FashionCLIP + BM25 text search + attributes (colour, type), TND budget, city · 3. top 5 with shop, price, city, match, link · 4. no result → M7's question.
Example	"white sneakers in Tunis under 150 TND" → Manuella, Basket femme, 86.4 TND, match 0.95.

LLM: turns the sentence into a query, writes the shortlist. Called by gap_filling with a structured call → 0.

photo_piece

Complete the look from a photo · "Should I buy?"

Actions	`complete_look(photo)` · `should_buy(photo, price_tnd)`
Workflow	1. photo from args or blackboard `last_photo` · 2. **FashionCLIP**: image vector + zero-shot type (t-shirt, jeans…) and colour · 3. the piece is added *virtually* to the wardrobe · 4. **M2 complete** around it → 5 outfits. **should_buy** adds: 5. duplicates = owned pieces of the same category with cosine > 0.9 · 6. good outfits = score ≥ 0.6 · 7. verdict: duplicate → Skip; ≥ 3 good outfits → Buy; else Think · 8. cost per wear = price / (10 × good outfits) (code).
Example	photo of a t-shirt, 45 TND → navy t-shirt, Buy, 5 good outfits, 0.9 TND per wear.

LLM: which action ("buy / prix" → should_buy) and the price, then the answer. Type and colour come from FashionCLIP, not the LLM.

declutter

Declutter suggestions

Actions	`suggest()`
Workflow	1. **M3 stats** → never worn; worn but not in 90 days · 2. near-duplicates: same category and FashionCLIP cosine > 0.93 → keep the most worn, let go the other.

LLM: only one action and no argument, so **routing is always skipped**; with a sentence, the LLM only writes the answer (1 call). Button → 0.

15. The two BO agents (orchestrators)

personalised_recommendations (BO1) · time_optimisation (BO2)

BO agents · BO2 = also "Quick natural-language request"

Knows	its own idea agents + the partner BO's idea agents (listed as `partner:agent`, e.g. `personalised_recommendations:forgotten`).
Workflow (sentence)	1. memory (taste learned if stated) · 2. **keyword rule**, else **LLM job 1** reading the cards → one agent · 3. its own idea → `ctx.call(agent, message)`: the idea agent's LLM picks the action (job 2) and runs its workflow, without writing · partner idea → the message goes to the **partner BO agent**, which hands it to its idea agent · 4. the BO agent's LLM writes the final answer from the facts (job 3) · 5. conversation saved.
Workflow (structured)	`"<agent>.<action>"` (e.g. `swap.swap`) or `"partner:agent.action"` → delegated directly: 0 LLM calls.
BO1 rules	laundry / dirty → BO2 laundry · "change the shoes" → BO2 swap · haven't worn → forgotten · pack / trip / je pars → travel · capsule · 3d / avatar · palette / drift → style_insights · same thing → rotation · missing → gap_filling · season · TND / shop → BO2 local_shops
BO2 rules	laundry / dirty / lent → laundry · swap → swap · haven't worn → BO1 forgotten · why → BO1 daily_outfit · pack / trip → travel · wedding / interview → event_looks · TND / shop → local_shops · photo → photo_piece · donate / declutter · today / tomorrow / widget → daily_plan
Default	BO1: daily_outfit · BO2: daily_plan (when nothing matches and the LLM's choice is invalid).

LLM: sentence → choose the agent (skipped if a rule matches) + write the final answer; the idea agent below uses its own LLM for the action: 2–3 calls in total. Structured → 0.

16. BO1 as a system: Personalised recommendations

Goal of the BO: outfits that fit *this* user: their clothes, their day, their taste, their history.

BO1 idea	Agent	Main tools
Daily outfit + Remix screen · Outfit builder · "Why this outfit"	daily_outfit	M4, M2
Rotation-aware penalty	rotation	M3, M2
Rediscover forgotten pieces	forgotten	M3, M2
Capsule wardrobe mode	capsule	M17, M4
Seasonal wardrobe refresh	seasonal_refresh	weather, M3, M2
3D avatar preview of the daily outfit	avatar_preview	M4, M9
Gap-filling suggestion	gap_filling	wardrobe analysis, local_shops (M7)
Travel outfit planner	travel (shared with BO2)	weather, M2, greedy
Personal colour palette · Style drift · Favourite looks / lookbook	style_insights	M3, colours

How the ideas connect: daily_outfit → (button) avatar_preview → 3D · daily_outfit.why explains any outfit on the blackboard, even one changed by BO2's swap · gap_filling → local_shops (BO2) · rotation and forgotten share M3's history with BO2's laundry (a piece in the laundry is never "forgotten" nor proposed).

17. BO2 as a system: Time optimisation

Goal of the BO: less time deciding: outfit ready before waking up, one-tap changes, no surprises (laundry, events), quick answers.

BO2 idea	Agent	Time saved by
Outfit of the day in one screen (cached) · Tomorrow's outfit the night before · Notification · Widget	daily_plan	computed in advance by M4 (05:00 / 20:00): 0 s at wake-up
One-tap swap of a single piece	swap	change one piece, keep the rest (0.15 s)
Laundry / unavailable UI	laundry	never proposed a dirty piece
Event countdown looks	event_looks	looks prepared days before
Local shops shortlist	local_shops	3–5 real local options instead of browsing
Complete the look from a photo · "Should I buy?"	photo_piece	decision in the shop in seconds
Declutter suggestions	declutter	a smaller wardrobe = faster choices
Packing outfits for a trip	travel (shared)	fewest pieces, outfit per day
Quick natural-language request	time_optimisation (the BO2 agent itself)	one sentence instead of navigating screens

Cross-BO in both directions: "show me pieces I haven't worn" sent to BO2 → BO2 agent → BO1 agent → forgotten. "change the shoes" sent to BO1 → BO1 agent → BO2 agent → swap. The user never needs to know which BO owns which idea.

18. Three full walkthroughs (real runs)

A. A button: "Change shoes" (app screen, BO2 route)

```
App sends:  {"agent": "time_optimisation", "payload": {"action": "swap.swap", "args": {"slot": "shoes"}, "reply":
false}}

time_optimisation   action named → no LLM (structured)                          ← 0 LLM
  └ agent swap.swap   action named → no LLM
       ├ wardrobe (RAG)          demo wardrobe, 67 pieces                       0.1 s
       ├ blackboard read         last_outfit = [red dress, burgundy shoes]
       ├ tool m02_recommendation.swap   slot shoes                              88 ms
       └ blackboard write        last_outfit = [red dress, olive shoes]
Result: data.before / data.after → the screen shows the pieces. LLM calls: 0. Total ≈ 0.15 s.
```

B. A sentence: "What should I wear tomorrow for work?" (BO1)

```
personalised_recommendations
  ├ memory                   2 past messages, profile {likes: [], dislikes: []}
  ├ LLM llm_route  (job 1)   cards of 16 agents → {"agent": "daily_outfit"}        7.5 s
  ├ agent daily_outfit       (message passed, reply:false)
  │    ├ LLM llm_route (job 2)  → action "recommend" (rule "wear" agrees)                    2.8 s
  │    │     day / occasion: the LLM's args, backed up by the sentence's words → tomorrow, work
  │    ├ tool m04_scheduler.calendar   tomorrow = work day                          4 ms
  │    ├ wardrobe (RAG) + dislikes removed
  │    ├ tool m02_recommendation.recommend   3 outfits, weather 22.7 °C            2.5 s
  │    └ blackboard write   last_outfits, last_day, last_occasion
  └ LLM llm_answer (job 3)   FACTS → "For tomorrow, you should wear a black t-shirt, a blue skirt, and
                             black shoes ... As an alternative, a light blue t-shirt, blue pants ..."   3.9 s
LLM calls: 3. Total ≈ 14 s. The pieces come from M2; the LLM chose the route and wrote the sentence.
```

C. Cross-BO: "Forgotten pieces" button sent to BO2

```
time_optimisation       payload "personalised_recommendations:forgotten.rediscover"  (0 LLM)
  └ agent personalised_recommendations   forgotten.rediscover  (0 LLM)
       └ agent forgotten   rediscover  (0 LLM)
            ├ wardrobe (RAG)
            ├ tool m03_wear_log.forgotten      light blue polo shirt: never worn        6 ms
            └ tool m02_recommendation.complete around it → + navy pants + white shoes   834 ms
3 agents, 2 BOs, 2 tools, 0 LLM calls, 0.9 s. With the sentence "show me pieces I haven't worn in a long
time", the BO2 rule "haven't worn" sends it the same way, then forgotten's LLM extracts days=45 and the
BO2 agent's LLM writes the answer (2 LLM calls).
```

19. Errors, questions and limits

- **An agent never crashes its caller.** Any error becomes `result.error` with the steps so far; a tool error (`ok: false`) is reported, not hidden.
- **Missing information → question**, not a guess: travel without a city, build without known pieces, event without a date, laundry without a piece, capsule plan without a capsule, photo actions without a photo. A sub-agent's question is passed up to the user.
- **Language:** French / Arabic detected by a word / script test; the answer is written in that language.
- **Limits seen:** 6–20 s per sentence on a 4 GB GPU; the 3B model sometimes mentions an alternative from an earlier turn. A 7B model (one setting: `LLM_LOCAL_MODEL`) or a cloud LLM improves both. Buttons are unaffected (no LLM).

20. Glossary

Agent	Program that receives a request, decides, uses tools, remembers, answers.
Orchestrator / BO agent	Agent whose tools are other agents; chooses who handles a request.
Idea agent	Agent for one BO idea (or a few close ones), with its own actions and LLM.
Tool / module agent	Exact code or model with named actions (M2, M3, M4, M7, M9, M17).
Action / workflow	A fixed procedure of an agent; the steps it runs.
Routing	Choosing the agent (BO agent) or the action + arguments (idea agent).
Structured call	A request that already names the action (button, other agent): no LLM needed.
JSON mode	The LLM can only output valid JSON (constrained decoding).
Guardrail	A keyword rule that overrides or locks the LLM's choice when the words are unambiguous.
RAG	Retrieve the facts first (wardrobe, history, calendar), then let the LLM use only them.
Grounding	The answer may only contain what is in the FACTS.
Blackboard	Shared short-term memory of a session that all agents read and write.
Zero-shot	Classifying with a pretrained model by comparing to text descriptions, without training.
Embedding / vector	A list of numbers representing an image or a text; close vectors = similar things.
ILP	Integer linear programming: exact optimisation (M17 capsule).
Agent card	The ID card of an agent (role, when to use, actions) read by other agents.
Handoff / next_actions	Buttons an agent proposes after its answer ("Show it on my avatar").